

A

PROJECT REPORT ON

**FPGA-BASED AHB-TO-APB BRIDGE USING
VERILOG HDL, INCORPORATING REDUCED
LATENCY**

SUBMITTED TO SAVITRIBAI PHULE PUNE UNIVERSITY
FOR PARTIAL FULFILLMENT
OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE OF

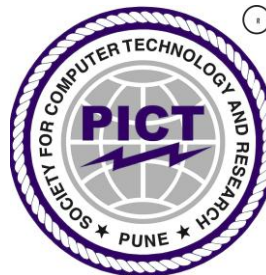
BACHELOR OF ENGINEERING
In
Electronics and Telecommunication Engineering

By

**SARTHAK CHOR
KARAN PATIL**

**B400050049
B400050204**

**NAME OF GUIDE
Mr. N. G. NIRMAL**



**DEPARTMENT OF
ELECTRONICS AND TELECOMMUNICATION ENGINEERING
PUNE INSTITUTE OF COMPUTER TECHNOLOGY
PUNE – 43**

Department of Electronics and Telecommunication Engineering
Pune Institute of Computer Technology, Pune – 43

CERTIFICATE

This is to certify that the Project Report entitled
**FPGA-Based AHB-to-APB Bridge Using Verilog HDL, Incorporating Reduced
Latency**
has been successfully completed by

Sarthak Chor B400050049

Karan Patil B400050204

Is a bona fide work carried out by them under the guidance of Mr. N. G. Nirmal and it is approved for the partial fulfillment of the requirement of the Savitribai Phule Pune University, Pune for the award of the degree of the Bachelor of Engineering (Electronics and Telecommunication Engineering). This project work has not been earlier submitted to any other Institute or University for the award of any degree or diploma.

Mr. N. G. Nirmal
Project Guide

Dr. G. S. Mundada
HOD, E&TCE Dept.

Dr. S.T. Gandhe
Principal, PICT

Place: Pune

Date :

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our project guide, Mr. N. G. Nirmal, for his continuous guidance, valuable suggestions, and support throughout the completion of this work. His technical insights and encouragement played a crucial role in shaping this project. We also extend our thanks to the Head of Department, Dr. G. S. Mundada, for providing the necessary resources and academic environment required for carrying out this work successfully. We are thankful to all the faculty members and staff of the department for their support and cooperation during this project. We would like to acknowledge our colleagues and friends for their assistance, discussions, and constant motivation, which helped us overcome various challenges during the course of this work. Finally, we acknowledge the use of reference materials, research papers, and other sources that have contributed to the understanding and development of this project.

Sarthak Chor

Karan Patil

CONTENTS

	Abstract	i
	List of Acronyms	ii
	List of Figures	iii
	List of Tables	iv
1	Introduction	1-11
1.1	Background	1
1.2	Relevance	3
1.3	Literature Survey	5
1.4	Motivation	7
1.5	Aim of the Project	9
1.6	Scope and Objectives	10
1.7	Technical Approach	11
2	AMBA Bus Architecture and System-Level Overview	13-21
2.1	Overview of AMBA Architecture	13
2.2	Advanced High-Performance Bus (AHB)	15
2.3	Advanced Peripheral Bus (APB)	17
2.4	Need for AHB-to-APB Bridge	19
2.5	Research Hypothesis and Problem Insight	20
3	System Design	22-33
3.1	Introduction to System Design	22
3.2	AHB Slave Interface	24
3.3	APB Master Interface	25
3.4	Address Decoder	26
3.5	FSM Controller Design	27
3.6	Data Path Design	30
3.7	Timing and Operation Analysis	31
3.8	Back-to-Back Transfer Mechanism	32
3.9	Power Optimization Implementation	33
4	Implementation, Testing and Debugging	34-39
4.1	Overview of FPGA-Based Implementation	34
4.2	Clock Divider Implementation	35
4.3	Driver Module Design	36
4.4	Scanner Module Design	36
4.5	State Observation Using LEDs	37

4.6	Top Module Integration	38
4.7	Testing and Debugging Approach	38
5	Results and Discussion	40-45
5.1	Write Transfer	40
5.2	Read Transfer	41
5.3	Latency Analysis	42
5.4	Timing Analysis	43
5.5	Resource Utilization Analysis	44
5.6	Power Consumption Analysis	45
6	Conclusions	46
7	Future Scope	47
	References	48-50

ABSTRACT

To sustain system performance in contemporary System-on-Chip (SoC) architectures, communication between high-speed cores and low-speed peripherals must be effective. Although the AHB and APB protocols are frequently used for this purpose, traditional AHB to APB bridges frequently cause latency because of numerous handshaking cycles, idle states, and ineffective clock management. The goal of this project is to create a low-latency, power-efficient AHB to APB bridge that enhances data transfer efficiency without sacrificing protocol accuracy. To eliminate unnecessary idle cycles and enable continuous transfers, the proposed design utilizes a compact finite state machine (FSM), early address decoding, and a one-cycle APB SETUP and one cycle ENABLE mechanism, reducing the transfer latency from 4 clock cycles in a conventional bridge to 3 clock cycles. This architectural optimization achieves 25% reduction in transaction latency. To lower power consumption without increasing wake up delay, a latency-aware fine-grain clock gating technique is used. The bridge is implemented in Verilog HDL and synthesized using the Xilinx Vivado design environment on a Spartan-7 FPGA platform. The resulting architecture guarantees dependable operation under a range of transfer conditions, efficient protocol conversion, and decreased latency.

Abbreviations and Acronyms

SoC	System on Chip
CPU	Central Processing Unit
GPIO	General Purpose Input/Output
AMBA	Advanced Microcontroller Bus Architecture
AHB	Advanced High-performance Bus
APB	Advanced Peripheral Bus
FPGA	Field Programmable Gate Array
HDL	Hardware Description Language
ASIC	Application-Specific Integrated Circuit
FSM	Finite State Machine
RTL	Register Transfer Level
VLSI	Very Large Scale Integration

List of Figures

Fig. 2.1	AMBA System Bus Hierarchy and SoC-Level Interconnect	13
Fig. 2.2	AHB Pipelined Transactions	15
Fig: 2.3	APB Non-Pipelined Transactions	17
Fig: 3.1	System Block Diagram of the Proposed Design.	22
Fig: 3.2	Finite State Machine of AHB-to-APB Bridge.	28
Fig: 4.1	Modified Design for FPGA Implementation	34
Fig: 5.1	Simulation Results of Write Transfer.	40
Fig: 5.1	Simulation Results of Read Transfer.	41

List of Tables

3.1	State transition table for the proposed FSM	29
5.1	Latency comparison between conventional and proposed AHB-to-APB bridge	42
5.2	Timing performance comparison between conventional and proposed AHB-to-APB bridge	43
5.3	Resource utilization comparison between conventional and proposed AHB-to-APB bridge	44
5.4	Timing performance comparison between conventional and proposed AHB-to-APB bridge	45

CHAPTER 1

Introduction

1.1 Background

The continuous evolution of semiconductor technology and digital system design has led to the development of highly integrated System-on-Chip (SoC) architectures. In modern SoCs, multiple functional units such as central processing units (CPUs), memory subsystems, digital signal processors, and a wide range of peripheral interfaces are integrated onto a single silicon platform. This high level of integration improves performance, reduces power consumption, and minimizes physical size, making SoCs essential for applications in embedded systems, consumer electronics, automotive systems, and communication devices.

As the complexity of these systems increases, efficient and reliable communication between different components becomes a major design challenge. Each subsystem within an SoC may operate at different clock frequencies, data rates, and power requirements. High-performance modules such as processors and memory demand fast and continuous data transfer, whereas peripheral devices like timers, serial communication modules, and general-purpose input/output (GPIO) units operate at lower speeds and prioritize simplicity and power efficiency. This mismatch in operational characteristics necessitates the use of structured and standardized communication protocols.

To address these challenges, ARM introduced the Advanced Microcontroller Bus Architecture (AMBA) in the late 1990s, which has since become one of the most widely adopted on-chip communication standards. AMBA provides a hierarchical bus structure that separates high-speed and low-speed communication domains, thereby improving scalability, modularity, and design reusability. Among its various protocols, the Advanced High-performance Bus (AHB) and the Advanced Peripheral Bus (APB) play a crucial role in many embedded system designs.

The AHB protocol is specifically designed for high-speed communication and supports features such as pipelined data transfer, burst transactions, and multiple bus

masters. These features enable efficient utilization of the bus and reduce idle cycles, making AHB suitable for connecting performance-critical components like processors and memory controllers. In contrast, the APB protocol is optimized for low-power and low-complexity operations. It follows a simple, non-pipelined two-phase transaction model consisting of SETUP and ENABLE phases, which simplifies hardware design and reduces power consumption, making it ideal for interfacing with low-speed peripheral devices.

However, due to the fundamental differences between AHB and APB protocols particularly in terms of timing, data transfer mechanisms, and control signalling direct communication between these two buses is not possible. AHB operates using a pipelined architecture where address and data phases overlap, while APB follows a sequential, non-pipelined approach. This difference creates a need for an intermediate component known as an AHB-to-APB bridge. The bridge acts as an interface that translates AHB transactions into corresponding APB operations while maintaining protocol compliance and ensuring data integrity.

Traditionally, AHB-to-APB bridges have been implemented using finite state machine (FSM)-based control logic to manage the transition between AHB and APB domains. These designs ensure correct operation but often introduce additional latency due to multiple clock cycles required for handshaking, state transitions, and idle conditions. In conventional implementations, a single data transfer may require four or more clock cycles, which can become a performance bottleneck in systems involving frequent peripheral access.

With the increasing demand for high-performance embedded systems, reducing communication latency has become a key design objective. Optimizing bridge architectures to support faster data transfer, reduced idle cycles, and efficient control flow can significantly enhance overall system responsiveness. Techniques such as early address decoding, optimized FSM design, and transaction overlap have emerged as effective approaches to improve bridge performance without compromising protocol correctness.

In recent years, Field Programmable Gate Arrays (FPGAs) have gained significant importance in the design and prototyping of digital systems. Unlike

Application-Specific Integrated Circuits (ASICs), FPGAs offer reconfigurability, faster development cycles, and cost-effective testing. Tools such as Xilinx Vivado provide a comprehensive design environment for synthesis, simulation, and implementation of hardware designs using Hardware Description Language s(HDLs) like Verilog. The Xilinx Spartan-7 FPGA platform, in particular, offers a balance between performance, power efficiency, and resource availability, making it suitable for implementing complex digital architectures such as AMBA-based bus systems.

Therefore, enhancing the efficiency of AHB-to-APB bridges by reducing latency and optimizing resource utilization is essential for improving the overall performance of SoC-based systems. This forms the foundation and motivation for the present project, which focuses on designing and implementing a low-latency, power-efficient AHB-to-APB bridge using Verilog HDL on a Xilinx Spartan-7 FPGA platform.

1.2 Relevance

The selected project, “FPGA-Based AHB-to-APB Bridge Using Verilog HDL, Incorporating Reduced Latency,” holds strong relevance to the field of Electronics and Communication Engineering (E&TC), as it directly addresses fundamental concepts of digital system design, embedded systems, and on-chip communication architectures. Modern E&TC engineering increasingly focuses on the design and optimization of complex integrated systems, where efficient data communication between different subsystems is a critical requirement.

One of the core areas of the E&TC curriculum is digital electronics and logic design, which forms the foundation for understanding hardware description languages such as Verilog. In this project, Verilog HDL is used to design and implement the bridge architecture, including modules such as the AHB interface, APB interface, address decoder, and finite state machine (FSM) controller. This directly reinforces concepts such as synchronous design, sequential circuits, state machine modeling, and RTL design.

The project is also closely related to the subject of microprocessors and microcontrollers, where students study system buses, memory interfacing, and peripheral communication. The AMBA protocol, including AHB and APB buses, is widely used in modern processor-based systems. Understanding how data is transferred between high-

speed processing units and low-speed peripherals through a bridge provides practical insight into real-world system design and processor architecture.

Another important domain covered in the E&TC curriculum is VLSI design and embedded system design. The implementation of an AHB-to-APB bridge reflects key VLSI concepts such as modular design, timing analysis, resource utilization, and power optimization. The use of an optimized FSM, early address decoding, and transaction overlap in the proposed design demonstrates how architectural-level decisions impact system performance, particularly in terms of latency reduction and throughput improvement.

The project also aligns with communication systems in a broader sense, as it focuses on reliable and efficient data transfer between different functional blocks within an integrated system. Although it operates at the chip level rather than over physical communication channels, the principles of data integrity, synchronization, and protocol conversion are analogous to those studied in communication engineering.

Furthermore, the use of FPGA technology, specifically the Xilinx Spartan-7 platform and Vivado design suite, makes this project highly relevant to practical engineering applications. FPGA-based implementation enables students to move beyond theoretical design and gain hands-on experience in synthesis, simulation, timing verification, and hardware validation. This exposure is essential for understanding real-world design constraints such as propagation delay, clock management, and hardware resource limitations.

Additionally, the project incorporates advanced optimization techniques such as fine-grain clock gating and latency-aware control logic, which are critical in modern low-power and high-performance system design. These concepts are increasingly important in industries such as IoT, embedded systems, and semiconductor design, where efficiency and performance must be carefully balanced.

Overall, this project integrates multiple core subjects of the E&TC curriculum, including digital design, microprocessors, VLSI design, and embedded systems, while also introducing practical implementation and optimization techniques. It provides a comprehensive learning experience that bridges theoretical knowledge with real-world

hardware design, making it highly relevant to both academic study and industry applications.

1.3 Literature Survey

The Advanced Microcontroller Bus Architecture (AMBA) has become a widely adopted standard for on-chip communication in System-on-Chip (SoC) designs due to its modularity, scalability, and support for reusable intellectual property (IP) cores. The AMBA framework was introduced to address challenges such as design complexity, lack of portability, inefficient custom bus implementations, and power management issues in embedded systems. By defining standardized interfaces and separating high-performance and low-power communication domains, AMBA enables efficient integration of heterogeneous components within a single chip [1].

Within AMBA, the Advanced High-Performance Bus (AHB) and Advanced Peripheral Bus (APB) serve distinct roles. AHB is designed for high-speed, high-bandwidth communication between processors, memory, and high-performance modules, while APB is optimized for low-power, low-complexity peripheral communication. The AHB-to-APB bridge acts as a critical interface that translates complex pipelined AHB transactions into simpler non-pipelined APB transactions, ensuring compatibility between these two domains. This bridge also handles clock domain synchronization, address decoding, and control signal generation, making it an essential component in modern SoC architectures [2], [3].

Early research focused on understanding AMBA protocols and ensuring correct implementation and verification of AHB and APB systems. Studies on AHB protocol design highlight its support for burst transfers, pipelined operations, and multi-master communication, which improve throughput but increase design complexity. Similarly, APB protocol studies emphasize its simplicity, low power consumption, and requirement of at least two cycles per transfer due to its non-pipelined nature. These fundamental differences necessitate efficient bridge architectures for seamless communication [4], [5].

Several works have explored the design and implementation of AHB-to-APB bridges using RTL methodologies. These designs typically employ finite state machine (FSM)-based control logic to manage transaction sequencing, including IDLE, SETUP,

and ENABLE states in APB communication. Such implementations ensure protocol compliance and correct signal translation between AHB and APB domains. However, they often rely on sequential processing, which introduces wait states and increases transaction latency [6], [7].

Power optimization has been a major focus in bridge design. Techniques such as clock gating, reset-aware architectures, and reduced switching activity have been proposed to minimize dynamic and static power consumption. For instance, incorporating reset controllers and power-aware FSM designs significantly reduces energy consumption during idle or low-activity periods. Additionally, scalable and low-power bridge architectures have been developed to support configurable data widths and multiple peripherals, achieving improvements in both power efficiency and bandwidth utilization [8], [9].

FPGA-based implementations have also been widely studied to evaluate timing performance and resource utilization. These works highlight the importance of clock management, synchronization, and critical path optimization in achieving efficient bridge operation. It has been observed that clock signals contribute significantly to dynamic power consumption, and techniques such as fine-grain clock gating can effectively reduce power usage while maintaining performance. However, these approaches primarily focus on power and timing optimization rather than latency reduction [10].

Area-efficient designs have been proposed to reduce hardware complexity and resource utilization. By optimizing data-path structures and simplifying control logic, these designs achieve reductions in silicon area and power consumption. For example, implementations based on updated AMBA specifications demonstrate measurable improvements in both area and power compared to earlier designs. Nevertheless, these designs strictly follow standard APB timing constraints, limiting opportunities for performance enhancement through techniques such as transaction overlap [11].

Recent research has explored enhancements in reliability and system-level efficiency. Integration of error detection and correction mechanisms into AHB-to-APB bridges improves data integrity and reliability in communication systems, especially in noise-prone environments. However, such approaches increase design complexity and may introduce additional latency overhead. Similarly, verification-oriented studies

emphasize the use of advanced methodologies such as SystemVerilog-based layered testbenches to ensure functional correctness and coverage of bridge designs [12], [13].

Beyond conventional bridge designs, studies on SoC communication efficiency highlight limitations in traditional bus-based architectures. It has been observed that standard AHB and APB interactions can lead to inefficient communication due to mismatches between pipelined and non-pipelined protocols. Advanced approaches propose alternative communication strategies that significantly reduce clock cycles and improve throughput, indicating the importance of optimizing interconnect design for performance [14].

Additionally, research on AHB bus monitoring and debugging introduces advanced techniques such as on-chip bus tracers with real-time compression and multi-resolution analysis. These approaches enable efficient debugging and performance analysis by capturing transaction-level and cycle-level data, highlighting the growing need for performance-aware design methodologies in complex SoCs [15].

From the reviewed literature, it is evident that existing AHB-to-APB bridge designs primarily focus on individual objectives such as power optimization, area reduction, scalability, or verification. While these approaches improve specific aspects of the system, limited work has been done on simultaneously optimizing latency, power, and efficiency within a unified framework. In particular, techniques such as early address decoding, optimized FSM transitions, and transaction overlap are not comprehensively explored together.

1.4 Motivation

With the increasing complexity of System-on-Chip (SoC) architectures, the demand for efficient communication between high-speed processing units and low-speed peripheral devices has become more critical than ever. Although the AMBA-based AHB-to-APB bridge is a widely adopted solution for interfacing these domains, existing implementations still suffer from certain limitations that restrict overall system performance.

From the study of existing literature and conventional designs, it is observed that most AHB-to-APB bridges primarily focus on ensuring protocol correctness and

functional reliability using finite state machine (FSM)-based control logic. While these designs successfully enable communication between AHB and APB buses, they often follow conservative state transitions and traditional handshaking mechanisms. As a result, additional clock cycles are introduced in the form of idle states and wait conditions, which increase the overall transaction latency.

Another key limitation identified in existing works is that optimization efforts are typically focused on either power reduction or area efficiency, rather than improving latency. Techniques such as clock gating and reduced switching activity have been effectively used to minimize power consumption; however, they do not directly address the delay associated with protocol conversion. Consequently, even optimized designs continue to exhibit relatively high latency during data transfers, especially in scenarios involving frequent peripheral access.

Furthermore, the mismatch between the pipelined nature of the AHB protocol and the non-pipelined operation of the APB protocol introduces additional inefficiencies. Since APB transactions are inherently sequential, conventional bridges often fail to fully utilize the advantages of AHB pipelining, leading to underutilization of system bandwidth. This gap highlights the need for design approaches that can better synchronize these two protocols and reduce unnecessary delays.

Another limitation lies in the lack of effective utilization of early signal processing techniques. In many traditional designs, address decoding and control signal generation occur after the completion of certain transaction phases, which delays the initiation of APB operations. This sequential dependency increases the number of clock cycles required for each transfer and reduces overall throughput.

Considering these limitations, there is a clear need for an improved AHB-to-APB bridge design that not only maintains protocol compliance but also enhances performance in terms of latency and efficiency. The motivation for this project arises from the objective to address these gaps by introducing architectural optimizations such as early address decoding, optimized FSM transitions, and support for back-to-back transactions. These techniques aim to minimize idle cycles, reduce control overhead, and enable faster data transfer between AHB and APB domains.

Additionally, the integration of fine-grain clock gating ensures that power

efficiency is preserved while improving performance. This balanced approach is important for modern embedded systems, where both speed and energy consumption are critical design considerations.

Therefore, the primary motivation behind this work is to develop a low-latency, power-efficient AHB-to-APB bridge that overcomes the limitations of conventional designs. By reducing the number of clock cycles required for data transfer and improving system responsiveness, the proposed solution contributes to more efficient SoC communication and better overall system performance.

1.5 Aim of the Project

The aim of this project is to design and implement a low-latency and power-efficient AHB-to-APB bridge using Verilog HDL on a Xilinx Spartan-7 FPGA platform. The project focuses on reducing the transaction latency associated with conventional AHB-to-APB bridges by optimizing the control architecture and data transfer mechanism.

The proposed work aims to minimize the number of clock cycles required for data transfer between the high-speed AHB bus and the low-speed APB bus while maintaining full AMBA protocol compliance. This is achieved by incorporating techniques such as early address decoding, optimized finite state machine (FSM) design, and support for back-to-back transactions.

Additionally, the project aims to improve overall system efficiency by integrating fine-grain clock gating to reduce unnecessary power consumption without affecting performance. The design is validated through simulation and implemented using Xilinx Vivado to ensure correct functionality and improved timing performance on FPGA hardware.

Thus, the primary goal is to develop an optimized AHB-to-APB bridge that enhances data transfer efficiency, reduces latency, and maintains a balance between performance, power, and hardware resource utilization, followed by FPGA prototyping using the Spartan-7 Boolean Board to validate the design through real-time hardware implementation and observation. This implementation also enables practical verification of control flow, timing behaviour, and system performance.

1.6 Scope and Objectives

1.6.1 Scope of the Project

The scope of this project is focused on the design, implementation, and validation of a low-latency AHB-to-APB bridge for efficient communication within System-on-Chip (SoC) architectures. The work primarily deals with protocol conversion between the high-speed AHB bus and the low-speed APB bus while ensuring compliance with AMBA specifications.

The design is implemented using Verilog HDL at the register-transfer level (RTL) and is targeted for FPGA realization using the Xilinx Vivado design environment on a Xilinx Spartan-7 platform. The scope includes simulation, functional verification, and timing analysis of the proposed architecture to evaluate its performance in terms of latency, operating frequency, and resource utilization.

The project specifically focuses on fixed-latency APB transactions, assuming peripherals with deterministic response time. Advanced features such as variable-latency peripheral support using PREADY, multi-master arbitration, and complex bus interconnects are beyond the scope of this work. However, the proposed design can be extended in future to support such features.

Additionally, the scope is limited to architectural and control-level optimizations aimed at reducing transaction latency and improving efficiency. Physical design aspects such as ASIC fabrication and large-scale SoC integration are not covered in this project.

1.6.2 Objectives of the Project

The main objective of this project is to design a functional AHB-to-APB bridge that ensures accurate protocol conversion between the AHB and APB buses while maintaining compliance with AMBA standards. A key focus of the work is to reduce transaction latency by optimizing the bridge architecture, with the aim of minimizing the number of clock cycles required for data transfer compared to conventional designs. To achieve this, techniques such as early address decoding are incorporated to enable faster initiation of APB operations and reduce delays between AHB and APB phases.

In addition, the project aims to develop an optimized finite state machine (FSM) that enables efficient control flow and supports back-to-back data transfers, thereby

improving overall throughput. Transaction overlap techniques are also employed to minimize idle cycles and enhance data transfer efficiency. Alongside performance improvements, fine-grain clock gating is implemented to reduce dynamic power consumption without compromising system functionality.

The design is developed using Verilog HDL and verified through simulation using the Xilinx Vivado design environment to ensure functional correctness. Further, the system is synthesized and implemented on a Xilinx Spartan-7 FPGA platform, where timing performance and hardware resource utilization are analyzed. Finally, the proposed design is evaluated in terms of latency, operating frequency, and overall efficiency improvements over conventional AHB-to-APB bridge architectures.

Overall, the objectives are directed toward developing a high-performance and efficient bridge architecture that overcomes the limitations of existing designs and enhances communication efficiency in modern SoC systems.

1.7 Technical Approach

The technical approach adopted in this project focuses on improving the efficiency of AHB-to-APB communication by reducing transaction latency while maintaining protocol correctness and power efficiency. Instead of redesigning the entire communication framework, the approach emphasizes optimizing key stages of the protocol conversion process to eliminate unnecessary delays and improve data transfer performance.

Initially, a detailed study of the AMBA AHB and APB protocols was carried out to understand their operational differences, particularly the mismatch between the pipelined nature of AHB and the non-pipelined behaviour of APB. Based on this analysis, the problem was identified as inefficient synchronization between the two protocols, leading to additional clock cycles during data transfer.

To address this, the approach focuses on minimizing idle cycles and improving control flow efficiency. One of the key strategies adopted is reducing dependency on sequential operations by preparing required signals in advance wherever possible. This helps in bridging the timing gap between AHB and APB transactions and enables faster initiation of peripheral access.

Another important aspect of the approach is optimizing control logic to support continuous data transfer. Instead of allowing the system to return to an inactive state after each transaction, the design is structured to handle successive transfers more efficiently. This reduces overhead associated with repeated initialization and improves overall throughput.

The approach also considers power efficiency alongside performance. Selective activation of logic elements is used to minimize unnecessary switching activity during inactive periods. This ensures that improvements in latency do not come at the cost of increased power consumption, maintaining a balanced design.

The implementation is carried out using Verilog HDL at the register-transfer level (RTL), allowing precise control over timing and hardware behaviour. Simulation and functional verification are performed using the Xilinx Vivado design environment to validate functional correctness and evaluate performance improvements.

Overall, the adopted technical approach is centered on optimizing timing, control flow, and signal handling within the bridge, rather than increasing hardware complexity. This enables the development of a solution that is efficient, scalable, and suitable for real-time SoC applications.

CHAPTER 2

AMBA Bus Architecture and System-Level Overview

2.1 Overview of AMBA Architecture

The Advanced Microcontroller Bus Architecture (AMBA) is a standardized on-chip communication framework developed by ARM Ltd. to address the growing complexity and integration requirements of modern System-on-Chip (SoC) designs. As semiconductor technology advances, multiple functional blocks such as processors, memory units, digital signal processors, and peripheral interfaces are integrated onto a single chip. Efficient communication between these heterogeneous components becomes a critical requirement for achieving high system performance, scalability, and reliability. AMBA provides a well-defined structure that enables seamless interaction between these components through standardized communication protocols.

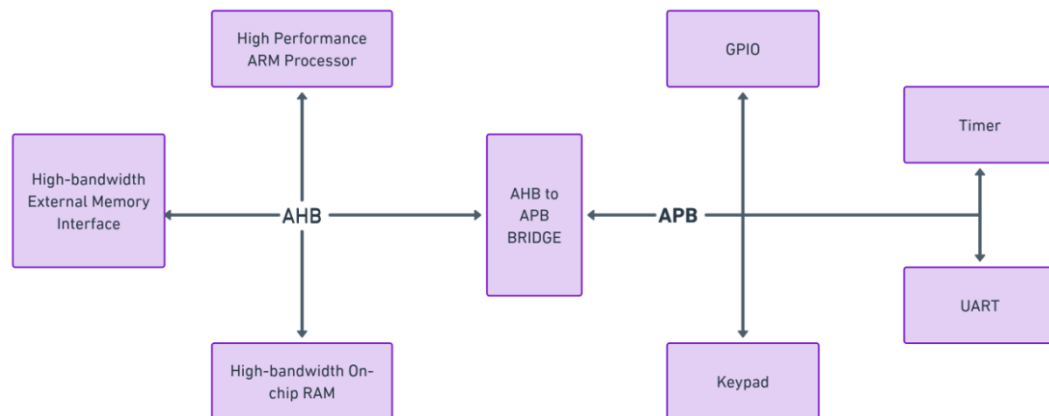


Fig: 2.1 AMBA System Bus Hierarchy and SoC-Level Interconnect.

Prior to the introduction of AMBA, SoC designs largely depended on custom or proprietary bus interfaces. These interfaces often resulted in increased design complexity, limited interoperability between components from different vendors, and reduced reusability of intellectual property (IP) cores. AMBA overcame these limitations by

introducing a uniform communication architecture, allowing designers to integrate pre-designed modules efficiently without extensive redesign. This standardization significantly reduces development time, simplifies verification, and enhances design flexibility.

A key feature of AMBA is its hierarchical bus architecture, which organizes system components based on their performance and power requirements. As illustrated in Fig. 2.1, high-performance modules such as processors, on-chip memory, and external memory interfaces are connected using high-speed buses, while low-speed peripheral devices are interfaced through simpler, low-power buses. This hierarchical separation ensures that high-speed data transfers are handled efficiently without being constrained by the limitations of slower peripherals. At the same time, low-power buses reduce unnecessary complexity and energy consumption in peripheral communication.

The AMBA architecture includes multiple bus protocols designed to serve different purposes within the system. The AMBA 2.0 specification introduced two widely used protocols: the Advanced High-Performance Bus (AHB) and the Advanced Peripheral Bus (APB). AHB is designed for high-speed communication and supports features such as pipelined operation, burst transfers, and multiple bus masters, making it suitable for connecting performance-critical components. In contrast, APB is a simpler, non-pipelined bus designed for low-bandwidth, control-oriented communication with peripheral devices, emphasizing low power consumption and ease of implementation.

Over time, AMBA has evolved to support more advanced system requirements. Newer protocols such as AXI (Advanced eXtensible Interface) and ACE (AXI Coherency Extensions) were introduced to provide higher bandwidth, improved scalability, and support for multi-core processing systems. These protocols enable more complex communication patterns and higher data throughput. However, despite these advancements, AHB and APB continue to be widely used in embedded systems due to their simplicity, deterministic timing behaviour, and efficient resource utilization.

The hierarchical organization of AMBA also enhances modularity and scalability in SoC design. Designers can independently develop and integrate different subsystems without affecting the overall architecture, making it easier to upgrade or modify specific

components. This modular approach is particularly beneficial in large-scale designs where flexibility and reusability are essential.

In summary, AMBA plays a fundamental role in modern digital system design by providing a scalable, modular, and efficient communication framework. Its hierarchical structure, combined with specialized bus protocols such as AHB and APB, enables optimized data transfer across different system domains. This makes AMBA a critical foundation for the development of high-performance and energy-efficient SoC architectures.

2.2 Advanced High-Performance Bus (AHB)

The Advanced High-Performance Bus (AHB) is a central component of the AMBA architecture, designed to provide high-speed communication between performance-critical modules within a System-on-Chip (SoC). It acts as the main system backbone, interconnecting components such as the central processing unit (CPU), memory subsystems, and high-speed peripherals. AHB is specifically optimized to deliver high data throughput, low latency, and efficient bus utilization, making it suitable for real-time and high-performance embedded applications.

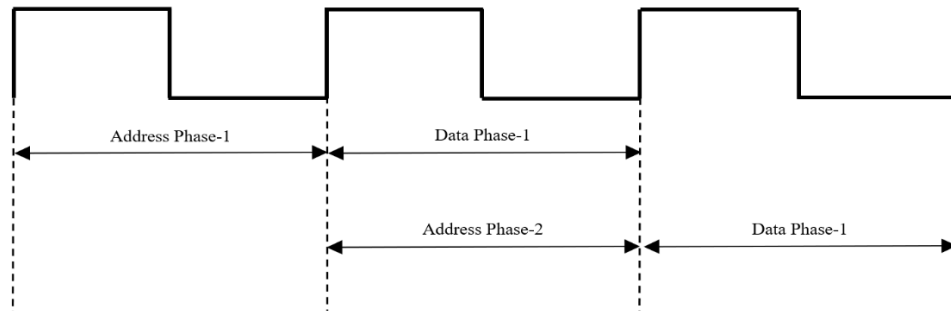


Fig: 2.2 AHB Pipelined Transactions.

A defining feature of AHB is its pipelined transaction mechanism, which significantly enhances system performance. In conventional bus systems, address and data transfers occur sequentially, leading to idle cycles and reduced efficiency. However, in AHB, the address phase and data phase of transactions are overlapped. As illustrated in Fig. 2.2, while the data phase of one transfer is being executed, the address and control

information for the next transfer are issued simultaneously. This overlapping of operations enables continuous data flow across the bus and minimizes idle time, thereby improving overall throughput.

Each AHB transfer consists of two main phases: the address phase and the data phase. During the address phase, the master places the address (HADDR) and control signals such as HWRITE (indicating read or write) and HTRANS (indicating the type of transfer) on the bus. In the subsequent clock cycle, the data phase begins, during which either write data (HWDATA) is sent to the slave or read data (HRDATA) is received from the slave. Due to pipelining, the address phase of the next transaction overlaps with the current data phase, as clearly shown in Fig. 2.2.

Another important capability of AHB is support for burst transfers. Burst mode allows a sequence of data transfers to occur consecutively without the need to repeatedly send address information for each transfer. This reduces overhead and increases efficiency, especially in applications involving large blocks of data such as memory operations. AHB supports different types of bursts, including single, incrementing, and wrapping bursts, providing flexibility in data transfer operations.

AHB also supports multiple bus masters, enabling different components such as processors and direct memory access (DMA) controllers to initiate transactions. To manage access to the shared bus, an arbitration mechanism is used to ensure that only one master controls the bus at a time. This arbitration process maintains orderly communication and prevents data conflicts, contributing to reliable system operation.

The protocol operates synchronously with a single clock, which simplifies timing analysis and ensures predictable behaviour. Key control signals such as HREADY and HRESP are used to manage handshaking and response status. The HREADY signal indicates whether the slave is ready to complete the current transfer or if wait states need to be inserted. The HRESP signal provides feedback about the success or failure of a transaction. These signals ensure proper synchronization between master and slave devices and maintain protocol compliance.

The pipelined structure shown in Fig. 2.2 plays a crucial role in reducing effective latency. By allowing consecutive transfers to overlap, the bus avoids unnecessary idle cycles that would otherwise occur in non-pipelined systems. This efficient utilization of

clock cycles makes AHB highly suitable for applications where high data throughput and low latency are essential.

In summary, the Advanced High-Performance Bus combines pipelined operation, burst transfer capability, multi-master support, and synchronous communication to deliver high-speed and efficient data transfer within an SoC. Its ability to overlap address and data phases, as depicted in Fig. 2.2, significantly improves system performance and makes it a vital component in modern embedded and VLSI system design.

2.3 Advanced Peripheral Bus (APB)

The Advanced Peripheral Bus (APB) is one of the fundamental bus protocols defined in the AMBA architecture, designed specifically for low-speed, low-power communication with peripheral devices. Unlike AHB, which is optimized for high-performance data transfer, APB focuses on simplicity, reduced hardware complexity, and energy efficiency. It is commonly used to interface with peripherals such as UARTs, timers, GPIO modules, and other control-oriented devices within a System-on-Chip (SoC).

APB operates using a non-pipelined transaction mechanism, which makes it simpler to design and implement compared to AHB. Each APB transfer is completed in a sequential manner without overlapping phases. This simplicity reduces control logic complexity and minimizes power consumption, making APB well-suited for peripheral communication where high throughput is not a primary requirement.

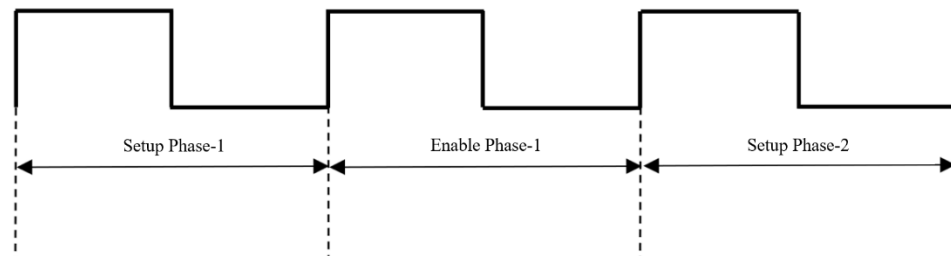


Fig: 2.3 APB Non-Pipelined Transactions.

A key characteristic of APB is its two-phase transfer protocol, consisting of the SETUP phase and the ENABLE phase. As illustrated in Fig. 2.3, during the SETUP phase, the master initiates the transfer by asserting the peripheral select signal (PSEL), providing the address (PADDR), and setting control signals such as PWRITE (indicating read or write operation) and PWDATA (write data, in case of write operations). These signals remain stable throughout the SETUP phase, preparing the system for data transfer.

In the following clock cycle, the ENABLE phase begins with the assertion of the PENABLE signal. This indicates that the transfer should be executed. During this phase, the peripheral either accepts the write data or provides read data through the PRDATA signal. The transaction is completed at the end of the ENABLE phase, after which the control signals can be de-asserted to return the bus to an idle state. As shown in Fig. 2.3, the clear separation between SETUP and ENABLE phases ensures deterministic and predictable timing behaviour.

Another important feature of APB is its simple one-to-many communication structure. A single APB master can control multiple peripheral slaves, with each slave being selected through an individual PSEL signal generated by an address decoder. Each peripheral is mapped to a fixed address region, which simplifies address decoding and reduces design complexity.

APB is designed to operate without complex handshaking mechanisms or pipelining, which further contributes to its low power consumption. Since signal transitions are limited and occur in a controlled manner, dynamic power usage is minimized. This makes APB particularly suitable for energy-efficient embedded systems where peripheral communication does not require high data rates.

However, the simplicity of APB comes with certain limitations. Due to its non-pipelined nature, it cannot support overlapping transactions, resulting in higher effective latency compared to AHB. Each transfer must be completed before the next one begins, which can become a bottleneck when frequent peripheral access is required.

In summary, the Advanced Peripheral Bus provides a simple, low-power, and reliable communication mechanism for connecting peripheral devices within an SoC. Its two-phase transfer protocol, as illustrated in Fig. 2.3, ensures clear and predictable

operation, while its reduced complexity makes it an efficient choice for low-speed applications. Despite its limitations in terms of throughput, APB remains an essential component of the AMBA architecture due to its effectiveness in peripheral interfacing. followed.

2.4 Need for AHB-to-APB Bridge

In modern System-on-Chip (SoC) architectures, different subsystems operate with varying performance, timing, and power requirements. The Advanced High-Performance Bus (AHB) is designed for high-speed, high-throughput communication, whereas the Advanced Peripheral Bus (APB) is intended for low-speed, low-power peripheral access. Due to these fundamental differences in design and operation, direct communication between AHB and APB is not feasible, which necessitates the use of an AHB-to-APB bridge.

One of the primary reasons for this requirement is the mismatch in transaction mechanisms. AHB follows a pipelined architecture where address and data phases overlap to improve throughput, as shown in Fig. 2.2, while APB operates using a non-pipelined, sequential two-phase transfer consisting of SETUP and ENABLE phases, as shown in Fig. 2.3. This difference creates a timing incompatibility, as AHB can initiate new transactions before the previous one completes, whereas APB requires each transfer to be completed before the next begins.

The AHB-to-APB bridge acts as an interface that resolves this mismatch by converting high-speed AHB transactions into corresponding APB transactions. From the perspective of the AHB bus, the bridge behaves as a slave device, while on the APB side, it functions as a master controlling peripheral devices. When an AHB master initiates a transfer targeting a peripheral address space, the bridge captures the address, control, and data signals, decodes the target peripheral, and generates the appropriate APB signals to complete the transaction.

In addition to protocol conversion, the bridge also performs timing synchronization between the two domains. Since APB operates at a slower rate, the bridge introduces wait states on the AHB side when required, ensuring that data is

transferred correctly without violating protocol constraints. This synchronization is essential to maintain data integrity and reliable system operation.

Furthermore, the bridge simplifies system design by isolating high-speed and low-speed communication domains. Instead of connecting peripherals directly to the high-performance bus, which would increase complexity and power consumption, the bridge allows peripherals to operate on a simpler and more efficient bus structure.

However, conventional AHB-to-APB bridges often introduce additional latency due to multiple control stages, idle states, and conservative state transitions. This can lead to inefficient utilization of system resources, particularly in applications involving frequent peripheral access. Therefore, improving the efficiency of the bridge, especially in terms of reducing latency, becomes an important design consideration.

In summary, the AHB-to-APB bridge plays a crucial role in enabling communication between heterogeneous bus systems within an SoC. It ensures protocol compatibility, manages timing differences, and maintains reliable data transfer, making it an essential component in AMBA-based system design.

2.5 Research Hypothesis and Problem Insight

Based on the analysis of AHB and APB protocols and the limitations of conventional bridge designs, it is observed that transaction latency is a major performance bottleneck in AHB-to-APB communication. Traditional bridge implementations typically require multiple clock cycles to complete a single transfer due to sequential control flow, delayed signal preparation, and the presence of idle states between transactions. This inefficiency becomes more significant in systems with frequent peripheral access, where cumulative latency can impact overall system performance.

The core problem identified is the inefficient synchronization between the pipelined AHB protocol and the non-pipelined APB protocol. While AHB is capable of initiating overlapping transactions, conventional bridges fail to fully utilize this capability, resulting in underutilization of available bandwidth. Additionally, in many designs, address decoding and control signal generation occur after certain phases of the transaction, which introduces avoidable delays.

The research hypothesis of this project is that the latency of an AHB-to-APB bridge can be reduced by optimizing the control logic and improving the timing of signal generation. Specifically, it is hypothesized that performing address decoding during the AHB address phase, rather than after it, can significantly reduce the delay in initiating APB transactions. By preparing control signals earlier, the bridge can transition more efficiently between AHB and APB operations.

Another key idea is to minimize unnecessary state transitions and idle cycles within the control mechanism. By designing an optimized control strategy that supports continuous data transfer and reduces dependency on sequential operations, it is possible to improve throughput and reduce overall latency. Enabling back-to-back transactions without returning to an idle state further enhances efficiency.

The hypothesis also considers maintaining a balance between performance improvement and hardware complexity. The proposed optimizations aim to reduce latency without significantly increasing resource utilization or violating AMBA protocol requirements. Additionally, power efficiency is taken into account by ensuring that performance enhancements do not lead to excessive switching activity.

Thus, the central idea of this work is that through careful optimization of timing, control flow, and signal handling, it is possible to reduce the transaction latency of an AHB-to-APB bridge by at least one clock cycle while maintaining protocol correctness and efficient resource usage. This forms the basis for the design and implementation presented in the subsequent chapters.

CHAPTER 3

System Design

3.1 Introduction to System Design

This section presents the complete system-level design and hardware realization of the proposed AHB-to-APB bridge. The design translates the conceptual low-latency methodology into a synthesizable register-transfer level (RTL) architecture using Verilog HDL. The implementation is targeted on a Xilinx Spartan-7 FPGA using the Xilinx Vivado design environment, enabling functional verification, synthesis, and timing validation of the proposed system.

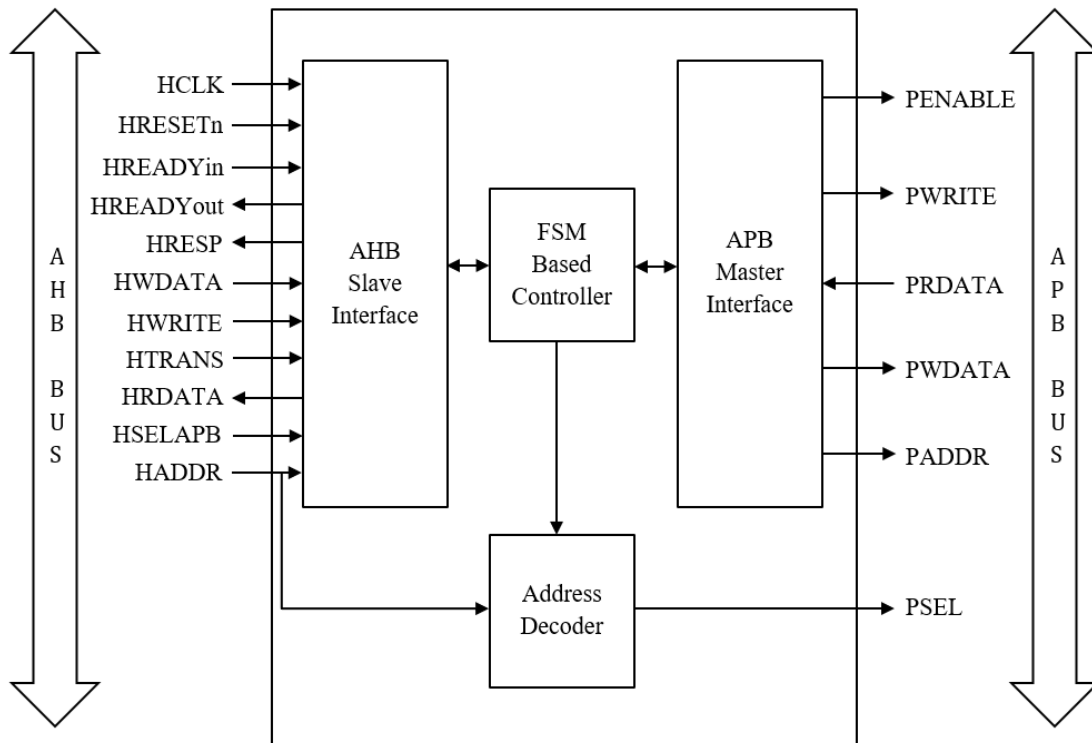


Fig: 3.1 System Block Diagram of the Proposed Design.

The overall architecture of the bridge is shown in Fig. 3.1. The design consists of four primary functional blocks: the AHB slave interface, the finite state machine (FSM)-based controller, the address decoder, and the APB master interface. These blocks are interconnected to enable seamless communication between the high-performance AHB bus and the low-power APB bus while maintaining protocol compliance and reducing transaction latency.

As shown in Fig. 3.1, the AHB slave interface acts as the entry point of the system and directly interacts with the AHB bus signals such as HADDR, HTRANS, HWRITE, HWDATA, HREADYin, and HSELAPB. This interface is responsible for capturing incoming transactions and forwarding relevant control and data information to the FSM controller. The FSM-based controller serves as the central control unit of the bridge, coordinating the sequence of operations and generating appropriate control signals for both AHB and APB domains.

The APB master interface, also illustrated in Fig. 3.1, is responsible for driving the APB signals such as PADDR, PWDATA, PWRITE, and PENABLE based on the control signals generated by the FSM. It ensures that APB transactions strictly follow the SETUP and ENABLE phases required by the APB protocol. The address decoder block receives the latched AHB address and generates the corresponding peripheral select signals (PSEL), enabling communication with the appropriate APB slave device.

The data flow within the system follows a structured path. During write operations, data from the AHB bus (HWDATA) is captured by the AHB slave interface and forwarded through the FSM controller to the APB master interface, which drives the data onto the APB bus (PWDATA). During read operations, data from the APB bus (PRDATA) is captured by the APB master interface and routed back through the bridge to the AHB bus as HRDATA. This bidirectional data transfer ensures proper communication between high-speed and low-speed components within the system.

Overall, as illustrated in Fig. 3.1, the proposed architecture integrates efficient control logic, early decision-making, and structured data flow to achieve low-latency operation while maintaining compatibility with AMBA protocols. The modular organization of the design also allows for scalability and ease of implementation on FPGA platforms.

3.2 AHB Slave Interface

The AHB slave interface serves as the primary point of interaction between the AHB bus and the proposed bridge. It is responsible for capturing incoming AHB transactions, validating them, and preparing the required signals for further processing within the bridge.

The interface captures key AHB signals including HADDR, HTRANS, HWRITE, and HWDATA. During the address phase of a transaction, HADDR, HTRANS, and HWRITE are monitored to determine the type and validity of the transfer, while HWDATA is captured during the data phase for write operations. These signals are synchronized with the system clock (HCLK) to ensure correct timing alignment.

A crucial part of the interface is the generation of the VALID signal, which determines whether an incoming transaction should be processed by the bridge. As implemented in the design, a transfer is considered valid when the following condition is satisfied:

$$\text{VALID} = \text{HSELAPB} \ \&\& \ \text{HREADYin} \ \&\& \ (\text{HTRANS} = \text{NONSEQ} \ \text{or} \ \text{SEQ})$$

This ensures that only active and intended AHB transfers targeting the APB domain are accepted, while idle or invalid transfers are ignored.

To maintain stability across clock cycles and ensure proper synchronization with the APB domain, the address is stored in a temporary register, HADDR_TEMP. This register holds the address throughout the transaction, allowing consistent access during both SETUP and ENABLE phases of the APB protocol.

The interface also manages the AHB handshake mechanism using HREADYin and HREADYout signals. The HREADYin signal indicates whether the AHB bus is ready to accept or provide data, while HREADYout is controlled by the bridge to indicate its readiness to complete a transaction. When the bridge is idle or ready for a new transfer, HREADYout is asserted.

During an ongoing APB transaction, the bridge may need to temporarily stall the AHB master to maintain proper synchronization between the two protocols. This is achieved by de-asserting HREADYout, effectively introducing wait states on the AHB

bus. Once the APB operation is completed, HREADYout is asserted again, allowing the AHB master to proceed with the next transaction.

Overall, the AHB slave interface ensures proper input capture, validation, and synchronization of signals, enabling smooth interaction between the AHB bus and the internal bridge logic.

3.3 APB Master Interface

The APB master interface is responsible for generating and controlling the signals required to perform transactions on the APB bus. It operates under the control of the FSM and ensures that all APB operations strictly follow the defined protocol timing.

The primary signals generated by the APB master interface include PADDR, PWDATA, PWRITE, and PENABLE. The PADDR signal carries the address of the target peripheral, while PWDATA carries the data to be written during write operations. The PWRITE signal indicates the type of operation, where a value of ‘1’ represents a write and ‘0’ represents a read. The PENABLE signal is used to distinguish between the SETUP and ENABLE phases of the APB transaction.

The operation of the APB interface is closely tied to the FSM states. During the READ and WRITE states, corresponding to the SETUP phase, the interface drives the address and control signals onto the APB bus while keeping PENABLE de-asserted. In the subsequent states, R_ENABLE and W_ENABLE, PENABLE is asserted, marking the ENABLE phase where the actual data transfer occurs. This direct mapping of FSM states to APB phases ensures correct timing and simplifies control logic.

For read operations, the interface places the address on PADDR and sets PWRITE to ‘0’ during the SETUP phase. In the ENABLE phase, the data provided by the APB peripheral on the PRDATA bus is captured and forwarded to the AHB side as HRDATA. This ensures correct data transfer from the APB domain to the AHB master.

For write operations, the interface places the address on PADDR and the data from HWDATA onto PWDATA during the SETUP phase, with PWRITE asserted. In the ENABLE phase, the data is written to the selected APB peripheral. This establishes a direct data path from the AHB bus to the APB bus.

The proposed design assumes fixed-latency APB transactions and does not include the PREADY signal. As a result, each APB transfer is completed in a deterministic number of clock cycles, simplifying control logic and reducing overall latency. This approach is suitable for peripherals with predictable response times and contributes to the efficiency of the bridge.

In summary, the APB master interface ensures proper signal generation, maintains protocol timing through FSM control, and facilitates reliable data transfer between the AHB and APB domains.

3.4 Address Decoder

The address decoder is responsible for translating the AHB address into corresponding APB peripheral select signals. In the proposed design, the decoder operates on the latched address stored in the register HADDR_TEMP, ensuring stable and consistent address information throughout the transaction. The decoded output is provided in the form of PSELx signals, where each bit corresponds to a specific APB peripheral. Thus, the mapping function can be represented as a direct relationship:

$$\text{HADDR_TEMP} \rightarrow \text{PSEL}_x$$

This mapping ensures that only the intended APB slave device is activated during a transaction, preventing unintended access to other peripherals.

A key feature of the proposed design is early address decoding. Instead of waiting for the completion of the AHB data phase, the address is decoded immediately after a valid transfer is detected, typically in the IDLE state before the APB SETUP phase begins. This allows the PSEL signal to be generated in advance, reducing the delay between the AHB address phase and the APB transaction. As a result, the bridge achieves faster response and reduced overall latency.

To improve power efficiency, clock gating is incorporated into the address decoding logic. The decoder operates on a gated clock signal generated as:

$$\text{Decoder Clock} = \text{AD_clk} \ \& \ \text{HCLK}$$

Here, the signal AD_clk is asserted only when a valid AHB transaction is detected. When no valid transfer is present, AD_clk is de-asserted, effectively disabling the decoder

clock. This prevents unnecessary switching activity within the decoder logic, thereby reducing dynamic power consumption.

Overall, the address decoder not only performs accurate address mapping but also contributes significantly to latency reduction and power optimization through early decoding and clock gating mechanisms.

3.5 FSM Controller Design

3.5.1 State Encoding

The control operation of the proposed AHB-to-APB bridge is governed by a finite state machine (FSM), as shown in Fig. 3.2. The FSM consists of five distinct states: IDLE, READ, R_ENABLE, WRITE, and W_ENABLE. These states are encoded using a 3-bit binary representation in accordance with the Verilog implementation. The state encoding is defined as follows:

- IDLE = 3'b000
- READ = 3'b001
- R_ENABLE = 3'b011
- WRITE = 3'b111
- W_ENABLE = 3'b110

Each state corresponds to a specific phase of operation within the bridge. The IDLE state represents the inactive condition where no valid transfer is taking place. The READ and WRITE states correspond to the SETUP phase of the APB protocol, while the R_ENABLE and W_ENABLE states correspond to the ENABLE phase. This structured encoding ensures clear separation between different operational phases and simplifies control signal generation.

3.5.2 State Transition Logic

The state transition behaviour of the FSM is determined by two primary signals: the VALID signal and the HWRITE control signal. The VALID signal indicates the presence of a valid AHB transaction, while HWRITE determines whether the operation is a read or a write.

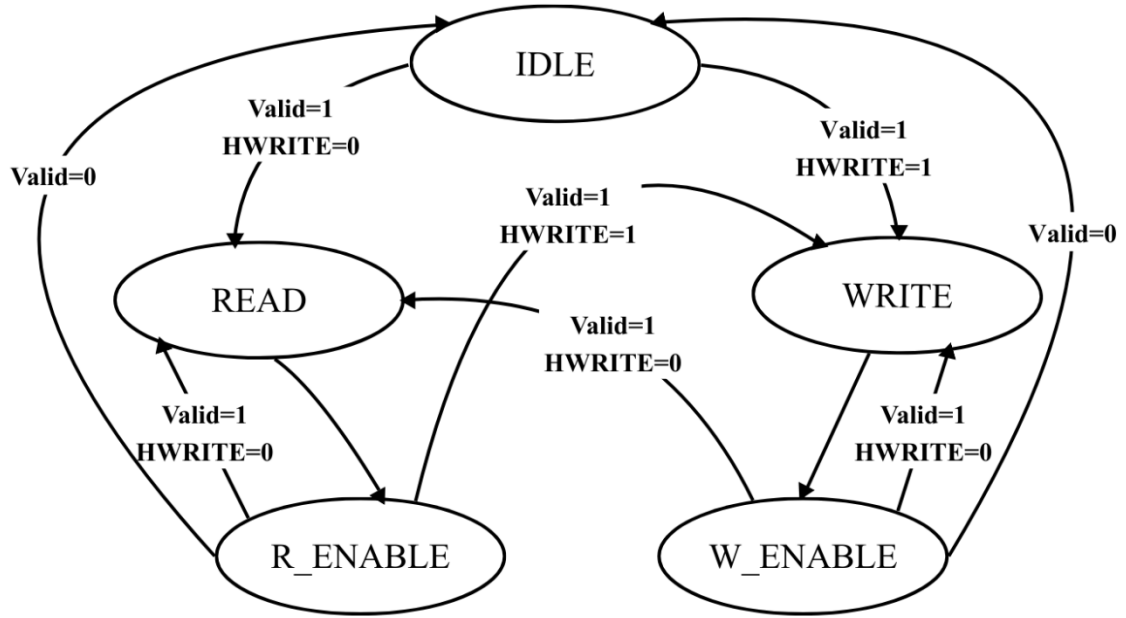


Fig: 3.2 Finite State Machine of AHB-to-APB Bridge.

As shown in Fig. 3.2 and summarized in Table 3.1, the FSM transitions from the IDLE state to either the READ or WRITE state based on the value of HWRITE when a valid transfer is detected. If $VALID = 1$ and $HWRITE = 0$, the FSM transitions to the READ state. If $VALID = 1$ and $HWRITE = 1$, it transitions to the WRITE state.

From the READ state, the FSM automatically transitions to the R_ENABLE state, and from the WRITE state to the W_ENABLE state, representing the progression from SETUP to ENABLE phase in the APB protocol. After completing the ENABLE phase, the FSM evaluates the input conditions again.

A significant feature of the proposed design is the support for back-to-back transactions. Instead of returning to the IDLE state after each transfer, the FSM checks for the presence of a new valid transfer at the end of the ENABLE phase. If $VALID$ remains high, the FSM directly transitions to the next READ or WRITE state depending on $HWRITE$. If no valid transfer is detected ($VALID = 0$), the FSM returns to the IDLE state.

This transition mechanism, clearly illustrated in Fig. 3.2 and Table 3.1, eliminates unnecessary idle cycles and improves overall throughput by enabling continuous data transfer between AHB and APB domains.

Table 3.1
State transition table for the proposed FSM

Current State	Condition	Next state
IDLE	Valid = 1 and HWRITE = 0	READ
IDLE	Valid = 1 and HWRITE = 1	WRITE
READ	-	R_ENABLE
WRITE	-	W_ENABLE
R_ENABLE	Valid = 1 and HWRITE = 1	WRITE
R_ENABLE	Valid = 1 and HWRITE = 0	READ
R_ENABLE	Valid=0	IDLE
W_ENABLE	Valid = 1 and HWRITE = 1	WRITE
W_ENABLE	Valid = 1 and HWRITE = 0	READ
W_ENABLE	Valid=0	IDLE

3.5.3 Output Control Logic

The output control logic of the FSM is derived directly from the current state of operation. Each state generates specific control signals required for proper communication with the APB bus.

In the READ state, corresponding to the SETUP phase of a read operation, the FSM de-asserts the PWRITE signal (PWRITE = 0) and prepares the address for the APB transfer. Similarly, in the WRITE state, the FSM asserts PWRITE signal (PWRITE = 1) and places valid write data on the PWDATA bus.

During the ENABLE states (R_ENABLE and W_ENABLE), the FSM asserts the PENABLE signal (PENABLE = 1), indicating that the APB transfer should be completed. This aligns with the APB protocol requirement where the ENABLE phase follows the SETUP phase to complete the transaction.

Thus, the control signal mapping can be summarized as:

- READ $\rightarrow$ PWRITE = 0
- WRITE $\rightarrow$ PWRITE = 1
- ENABLE states $\rightarrow$ PENABLE = 1

This direct relationship between FSM states and output signals simplifies control logic implementation and ensures strict adherence to APB timing requirements. The FSM-based control structure, as depicted in Fig. 3.2, enables efficient and deterministic operation of the bridge while supporting low-latency data transfer.

3.6 Data Path Design

In the proposed AHB-to-APB bridge, the design clearly separates the control path and the data path to achieve efficient operation and reduced hardware complexity. While the FSM governs the sequencing and control signals, the data path is responsible for actual data transfer between the AHB and APB domains, as reflected in the architecture shown in Fig. 3.1.

For write operations, data originates from the AHB master on the HWDATA bus. This data is captured during the AHB data phase and held stable as the FSM transitions to the WRITE state, which corresponds to the APB SETUP phase. In this phase, the write data is transferred to the APB domain through the PWDATA signal. In the subsequent W_ENABLE state (ENABLE phase), the data is written to the selected APB peripheral. Therefore, the write data path is directly mapped as:

$$\text{HWDATA} \rightarrow \text{PWDATA}$$

This direct mapping minimizes intermediate storage and reduces propagation delay, contributing to lower latency.

For read operations, the data flow occurs in the reverse direction. During the ENABLE phase (R_ENABLE state), the selected APB peripheral places valid data on the PRDATA bus. This data is immediately captured and forwarded to the AHB side through the HRDATA signal. Hence, the read data path is:

$$\text{PRDATA} \rightarrow \text{HRDATA}$$

This ensures that data from the APB domain is efficiently delivered to the AHB master without additional buffering stages.

A critical element in the data path is the temporary address register HADDR_TEMP. As implemented in the design, the address is latched during the AHB address phase and stored in HADDR_TEMP. This ensures that the address remains stable across both SETUP and ENABLE phases of the APB transaction, as shown in Fig. 3.2. This decoupling of address timing between AHB and APB domains is essential for correct synchronization and prevents timing violations.

Signal synchronization across the design is achieved using the common clock signal (HCLK). All data transfers and register updates occur at defined clock edges, ensuring deterministic behaviour. The FSM coordinates the timing of data movement so that data is transferred only when valid and stable, maintaining data integrity across the bridge.

Overall, the optimized data path design reduces unnecessary register stages, minimizes delay, and supports efficient bidirectional data transfer between AHB and APB buses.

3.7 Timing and Operation Analysis

The timing behaviour of the proposed AHB-to-APB bridge is governed by the FSM transitions shown in Fig. 3.2 and the state transition conditions summarized in Table 3.1. The design ensures that each transaction follows a structured sequence while minimizing unnecessary clock cycles.

For a read operation, the process begins with the AHB address phase, where a valid transfer is detected based on the VALID condition. Once detected, the FSM transitions from the IDLE state to the READ state, which corresponds to the APB SETUP phase. In this phase, the address is placed on PADDR and PWRITE is de-asserted. In the next clock cycle, the FSM moves to the R_ENABLE state, representing the ENABLE phase, where PENABLE is asserted and data from the APB peripheral is captured via PRDATA and transferred to HRDATA. Thus, the sequence is:

AHB_Address → READ → R_ENABLE → Data_Transfer

For a write operation, a similar sequence is followed. After detecting a valid transfer, the FSM transitions from IDLE to WRITE, where the address and data are placed on

PADDR and PWDATA respectively. This corresponds to the SETUP phase. In the next cycle, the FSM transitions to W_ENABLE, where PENABLE is asserted and the data is written to the APB peripheral. The sequence is:

AHB_Address → WRITE → W_ENABLE → Write_Transfer

As evident from Fig. 3.2, the FSM avoids unnecessary intermediate states, ensuring that each transaction is completed in a minimum number of clock cycles. Additionally, early address decoding (performed before the SETUP phase) reduces the delay between AHB and APB operations.

The absence of redundant IDLE transitions and the direct progression from SETUP to ENABLE phases contribute significantly to latency reduction. Compared to conventional designs, the proposed bridge reduces the number of clock cycles required per transaction, thereby improving overall throughput while maintaining protocol compliance.

3.8 Back-to-Back Transfer Mechanism

A major contribution of the proposed design is the implementation of an efficient back-to-back transfer mechanism, which significantly enhances system performance. In conventional AHB-to-APB bridges, the FSM typically returns to the IDLE state after each transaction, introducing idle cycles that increase latency.

In contrast, the FSM in the proposed design, as shown in Fig. 3.2 and detailed in Table 3.1, allows direct transitions between ENABLE states and subsequent transfer states when a valid transaction is detected. Specifically:

- From R_ENABLE → READ or WRITE
- From W_ENABLE → READ or WRITE

These transitions are determined by the VALID signal and the HWRITE control signal. If VALID = 1, the FSM immediately proceeds to the next READ or WRITE state without entering IDLE. Only when VALID = 0 does the FSM return to the IDLE state.

This mechanism enables continuous transaction flow, eliminating idle cycles between consecutive operations. As a result, the bridge can handle successive read and write requests efficiently without interruption.

The back-to-back transfer capability directly contributes to reduced latency and improved throughput. It ensures better utilization of both AHB and APB buses and aligns with the design objective of minimizing clock cycle overhead. This feature, clearly illustrated in Fig. 3.2 and Table 3.1, represents a key improvement over conventional bridge architecture.

3.9 Power Optimization Implementation

The proposed bridge incorporates power optimization techniques to reduce dynamic power consumption while maintaining performance. One of the primary techniques used is fine-grain clock gating applied to the address decoder, as part of the overall architecture.

In the design, the address decoder is driven by a gated clock signal generated using the control signal AD_clk. The effective clock supplied to the decoder is:

$$\text{Clock_decoder} = \text{AD_clk} \ \&\& \ \text{HCLK}$$

The AD_clk signal is asserted only when a valid AHB transaction is detected. During this period, the decoder is active and performs address mapping to generate the appropriate PSEL signals. When no valid transfer is present (i.e., during idle conditions), AD_clk is de-asserted, effectively disabling the clock to the decoder.

This selective activation prevents unnecessary switching activity within the decoder logic, which is a major source of dynamic power consumption in digital circuits. By disabling the decoder during inactive periods, the design reduces power usage without affecting functionality.

Furthermore, since the clock gating is applied only to the address decoder and not to critical control paths, the overall system maintains timing integrity and protocol compliance. The decoder is always enabled when required, ensuring correct operation.

Thus, the integration of clock gating using AD_clk achieves an effective balance between power efficiency and performance, making the proposed design suitable for low-power SoC applications.

CHAPTER 4

Implementation, Testing and Debugging

4.1 Overview of FPGA-Based Implementation

The proposed AHB-to-APB bridge is implemented on a Xilinx Spartan-7 FPGA platform to validate its functionality in a real hardware environment. While the design has been verified at the simulation level, FPGA implementation provides an additional level of confidence by demonstrating correct operation under real-time conditions, including clock behaviour, signal propagation, and hardware constraints.

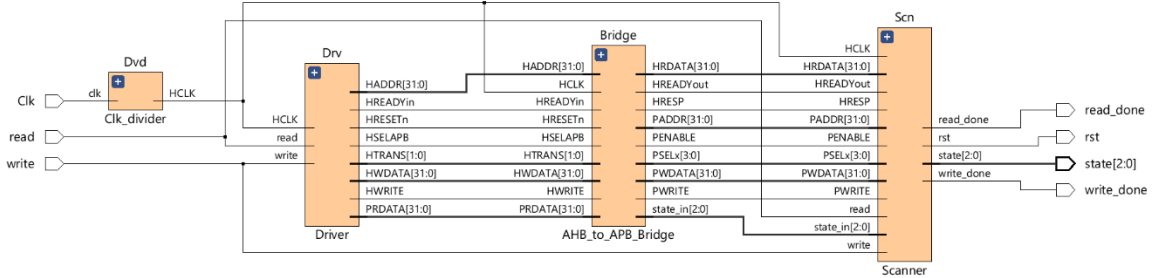


Fig: 4.1 Modified Design for FPGA Implementation.

The bridge design follows AMBA protocol specifications and involves a large number of input and output signals, including address, data, control, and handshake signals. However, directly interfacing all these signals with physical switches and LEDs available on the FPGA board is not feasible due to hardware limitations and usability constraints. Manually controlling such a large number of signals would also make testing complex and error-prone.

To address this issue, as shown in Fig. 4.1, an auxiliary testing framework is developed around the core bridge module. This framework includes additional modules such as a clock divider, driver module, and scanner module. These modules simplify interaction with the bridge by reducing the number of external inputs and outputs while still preserving the functional behaviour of the system.

The overall implementation approach ensures that the bridge can be tested using minimal user inputs while internally maintaining full protocol-compliant signal

generation and processing. This allows the system to behave as a realistic AHB-to-APB interface while being practically operable on FPGA hardware.

The FPGA implementation also enables observation of internal states and outputs using LEDs, providing a visual understanding of system behaviour. This is particularly useful for debugging and demonstration purposes, as it allows verification of state transitions, data transfer correctness, and control flow in real time.

4.2 Clock Divider Implementation

The Spartan-7 FPGA board provides a default clock input of 100 MHz, which is suitable for high-speed digital operation but too fast for direct observation of internal activities such as FSM state transitions. At this frequency, state changes occur within nanoseconds, making it impossible to visually track the behaviour using LEDs.

To address this limitation, a clock divider module is implemented to reduce the effective clock frequency driving the system. The divider uses a counter-based approach, where a 32-bit counter increments on every rising edge of the input clock. When the counter reaches a predefined value of 50,000,000, the output clock signal (HCLK) toggles and the counter resets.

Since the input clock frequency is 100 MHz, each clock cycle corresponds to 10 ns. Therefore, counting 50,000,000 cycles results in a time duration of 0.5 seconds for one toggle. As the output clock completes one full cycle (HIGH and LOW) in two such toggles, the total period of the output clock becomes 1 second. This results in an effective output frequency of 1 Hz.

The divided clock is then used to drive the bridge and associated modules. This significantly slows down the system operation, allowing each FSM state transition to be observed clearly on the LEDs. As a result, the sequence of operations such as IDLE, READ, ENABLE, and WRITE states can be visually tracked in real time.

It is important to note that the clock divider is used only for demonstration and debugging purposes. It does not affect the logical functionality or correctness of the bridge design. Instead, it enhances visibility and makes the system behaviour easier to analyse during FPGA-based testing.

4.3 Driver Module Design

The driver module is designed to generate valid AHB and APB signals required to stimulate the bridge during FPGA testing. In a typical system, these signals would be generated by a processor or bus master; however, for standalone testing on FPGA, a simplified method is required.

The driver module reduces the complexity of input control by using only two external signals: read and write. These signals act as triggers for generating complete AHB transactions internally. When the read input is activated, the driver configures the system to perform a read operation by setting HWRITE to logic low and generating valid control signals such as HSELAPB, HREADYin, and HTRANS. A fixed address is assigned to HADDR, and corresponding data is provided on the PRDATA bus to simulate a peripheral response.

Similarly, when the write input is activated, the driver generates a write transaction by setting HWRITE to logic high and assigning appropriate values to HADDR and HWDATA. The HTRANS signal is set to indicate a valid transfer, ensuring that the bridge processes the transaction correctly.

The driver ensures that all generated signals comply with AMBA protocol requirements, including valid transfer indication and proper control sequencing. By encapsulating the complexity of signal generation within a single module, the driver allows the user to interact with the system using only two switches, significantly simplifying the testing process.

This approach enables consistent and repeatable testing conditions, ensuring that the bridge operates under well-defined scenarios without requiring manual configuration of multiple signals.

4.4 Scanner Module Design

The scanner module is responsible for monitoring the outputs of the bridge and determining whether the executed operations are successful. Given the large number of output signals generated by the bridge, directly observing all of them is impractical. Therefore, the scanner condenses the output information into a few meaningful

indicators.

The scanner continuously evaluates key signals from both AHB and APB domains, including HRDATA, PWDATA, PWRITE, PSELx, and PADDR. For a read operation, it verifies whether the data received on HRDATA matches the expected value provided by the driver through PRDATA. It also checks that the operation type (PWRITE = 0), address, and peripheral selection signals are correct. If all conditions are satisfied, the scanner asserts the read_done signal.

For write operations, the scanner checks whether the data on PWDATA matches the expected value from HWDATA and verifies control signals such as PWRITE, PSELx, and PADDR. If the write operation is successfully executed, the write_done signal is asserted.

Additionally, the scanner generates a reset indication (rst) when neither read nor write operation is active. This helps identify idle conditions and ensures that the system starts from a known state.

By converting complex signal interactions into simple output flags, the scanner provides a clear and concise way to verify system functionality. This significantly simplifies debugging and enhances the usability of the FPGA implementation.

4.5 State Observation Using LEDs

To provide visibility into the internal operation of the bridge, a 3-bit state output is used to represent the current state of the FSM. This state information is mapped to LEDs on the FPGA board, allowing real-time observation of state transitions. Each state is encoded as a unique 3-bit value:

- 000 represents IDLE state
- 001 represents READ state
- 010 represents R_ENABLE state
- 011 represents WRITE state
- 100 represents W_ENABLE state

These values are continuously updated based on the FSM transitions within the bridge. Due to the reduced clock frequency provided by the clock divider, the transitions between these states occur slowly enough to be observed visually.

This feature is particularly useful for debugging, as it allows users to track the progression of operations step by step. By observing the sequence of state changes, one can verify whether the bridge is following the correct operational flow for read and write transactions.

The LED-based state display thus serves as an effective debugging and demonstration tool, providing insight into the internal working of the system without requiring complex measurement equipment.

4.6 Top Module Integration

The FPGA_Top module integrates all individual components into a complete system for implementation and testing. It acts as the top-level module that connects the clock divider, driver module, AHB-to-APB bridge, and scanner module.

The clock divider generates the system clock (HCLK), which is distributed to all modules to ensure synchronized operation. The driver module receives user inputs and generates corresponding AHB and APB signals. These signals are processed by the bridge, which performs protocol conversion and data transfer. The scanner module monitors the outputs and generates simplified status signals.

This modular structure ensures clear separation of functionality and makes the design easier to understand, test, and debug. Each module performs a specific role, and their integration results in a complete and functional system.

The top module also connects input switches and output LEDs through FPGA constraints, enabling direct interaction with the hardware. This integration allows the entire system to operate cohesively, demonstrating the functionality of the proposed bridge in a real-world environment.

4.7 Testing and Debugging Approach

The testing of the implemented design is carried out by activating the read and write inputs and observing the corresponding outputs on the FPGA board. When a read operation is initiated, the system transitions through the appropriate FSM states, and the read_done signal is asserted upon successful completion. Similarly, for write operations,

the write_done signal indicates successful execution.

Debugging is performed by observing both the state transitions and output signals. The LED-based state display provides insight into the internal operation of the FSM, allowing identification of incorrect transitions or unexpected behaviour. The scanner outputs further help in verifying data correctness and control signal integrity.

Any discrepancy in expected behaviour can be traced by analysing the sequence of states and signal values, making it easier to locate and correct errors. This approach ensures thorough validation of both functional and performance aspects of the design.

Overall, the FPGA implementation, combined with simplified input/output handling and visual debugging tools, provides an effective platform for testing and validating the proposed AHB-to-APB bridge design.

CHAPTER 5

Results and Discussion

5.1 Write Transfer

The simulation waveform of the write operation as shown in the Fig. 5.1 validates the correct functioning of the proposed AHB-to-APB bridge and clearly demonstrates the optimized low-latency behaviour described in the research work.

Initially, the system remains in the IDLE state, where no valid transfer is present. When a valid AHB write transaction is initiated (HSELAPB = 1, HREADYin = 1, and HTRANS indicating a valid transfer with HWRITE = 1), the FSM transitions directly from IDLE to the WRITE state. As observed in the waveform, the address HADDR is captured and held in the internal register HADDR_TEMP, and the write data HWDATA is prepared for transfer.

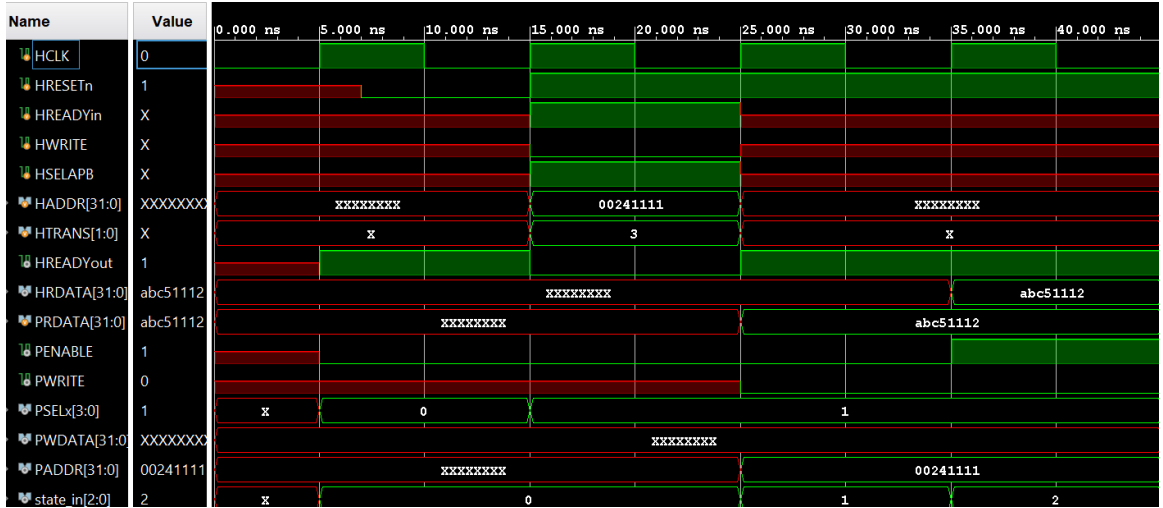


Fig: 5.1 Simulation Results of Write Transfer.

During the WRITE state, which corresponds to the APB SETUP phase, the bridge drives the APB signals such that PADDR is assigned the decoded address and PWDATA carries the write data. The control signal PWRITE is asserted, while PENABLE remains low, indicating that the transaction is in the setup phase. The waveform confirms that both address and data remain stable during this period.

In the next clock cycle, the FSM transitions to the W_ENABLE state, where

PENABLE is asserted to complete the APB transaction. The waveform shows that PSELx is activated appropriately and the write operation is successfully executed in this ENABLE phase. Importantly, there are no unnecessary intermediate or idle states between WRITE and W_ENABLE, confirming the optimized FSM behaviour.

The proposed design achieves latency reduction by eliminating redundant wait states and enabling efficient state transitions. In conventional designs, a write operation typically requires 4 clock cycles due to additional control overhead. However, in the proposed architecture, the write transaction is completed in 3 clock cycles (IDLE → WRITE → W_ENABLE), thereby reducing latency by one clock cycle.

This improvement is achieved through early address decoding and streamlined FSM control, both of which are evident in the waveform where the APB signals are prepared without delay. The simulation results therefore validate the effectiveness of the proposed low-latency design methodology.

5.2 Read Transfer

The simulation waveform of the read operation as shown in the Fig. 5.2 demonstrates correct data transfer from the APB domain to the AHB domain while maintaining protocol compliance and optimized timing behaviour.

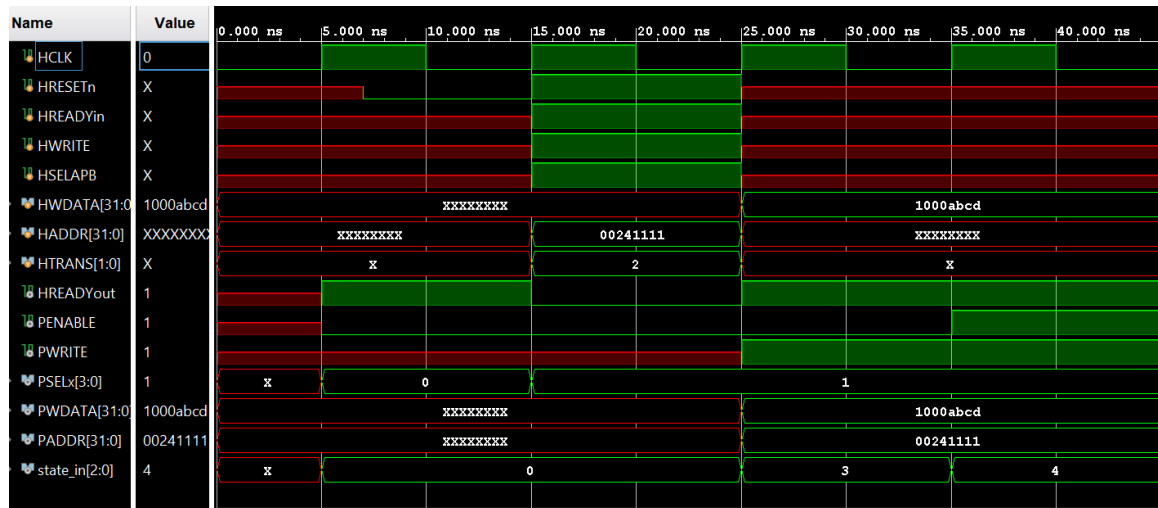


Fig: 5.1 Simulation Results of Read Transfer.

At the beginning, the system is in the IDLE state. Upon detection of a valid read request (HSELAPB = 1, HREADYin = 1, HTRANS valid, and HWRITE = 0), the FSM

transitions from IDLE to the READ state. As observed in the waveform, the address HADDR is latched and forwarded to the APB interface as PADDR.

During the READ state, corresponding to the APB SETUP phase, the bridge configures control signals such that PWRITE is de-asserted, indicating a read operation. The peripheral select signal PSELx is activated, while PENABLE remains low. The waveform confirms that the address and control signals are stable during this phase.

In the following clock cycle, the FSM transitions to the R_ENABLE state, where PENABLE is asserted to initiate the data transfer. At this stage, the APB peripheral provides valid data on the PRDATA bus, which is captured and transferred to the AHB side through HRDATA. The waveform clearly shows alignment between PRDATA and HRDATA during this phase, confirming correct data propagation.

The state progression observed in the waveform (IDLE → READ → R_ENABLE) matches the FSM design and demonstrates efficient execution without unnecessary idle cycles. Similar to the write operation, the read transaction is also completed within three clock cycles, reflecting the optimized control flow of the proposed design.

The absence of additional wait states and the direct transition between states highlight the effectiveness of early address decoding and improved FSM logic. These features ensure that the bridge achieves reduced latency while maintaining reliable and deterministic data transfer between AHB and APB domains.

5.3 Latency Analysis

Latency is a critical performance metric in bus bridge design, as it directly impacts data transfer efficiency. The proposed AHB-to-APB bridge is designed to reduce latency by optimizing FSM transitions and enabling early address decoding.

Table 5.1
Latency comparison between conventional and proposed AHB-to-APB bridge

Conventional Design	Proposed Design
4 clock cycles	3 clock cycles

As shown in Table 5.1, the conventional AHB-to-APB bridge requires 4 clock cycles to complete a write transaction due to additional wait states and sequential control flow. In contrast, the proposed design completes the same operation in 3 clock cycles by eliminating unnecessary idle states and enabling direct transitions between FSM states.

This reduction of one clock cycle per transaction significantly improves system throughput, especially in applications involving frequent peripheral access. The improvement is achieved without compromising protocol correctness or system stability. The reduced latency is primarily attributed to:

- Early address decoding during the AHB address phase
- Optimized FSM transitions
- Support for back-to-back transactions without returning to IDLE

These enhancements validate the effectiveness of the proposed low-latency design methodology.

5.4 Timing Analysis

The timing performance of the proposed design is evaluated and compared with the conventional AHB-to-APB bridge, as summarized in Table 5.2. The results indicate a noticeable improvement in maximum operating frequency and timing margins.

The proposed design supports a higher maximum frequency of 166.7 MHz compared to 142.86 MHz in the conventional design. This improvement indicates better optimization of the critical path and more efficient signal propagation.

Table 5.2
Timing performance comparison between conventional and proposed AHB-to-APB bridge

Parameter	Conventional	Proposed
Max Frequency Supported	142.86 MHz	166.7 MHz
Worst Negative Slack at Max Frequency	0.510 ns	0.315 ns
Worst Hold Slack at Max Frequency	0.132 ns	0.132 ns
Worst Pulse Width Slack at Max Frequency	3.000 ns	2.500 ns

The worst negative slack (WNS), which reflects timing margin, is also improved from 0.510 ns in the conventional design to 0.315 ns in the proposed design. This indicates that the design meets timing constraints more comfortably and can operate reliably at higher frequencies.

The hold slack remains unchanged, ensuring stable data retention across clock edges. Although there is a slight reduction in pulse width slack, it remains within acceptable limits and does not affect functionality. Overall, the timing analysis confirms that the proposed design achieves better performance while maintaining timing integrity and reliability.

5.5 Resource Utilization Analysis

The resource utilization of the proposed design is analyzed and compared with the conventional implementation, as shown in Table 5.3. The results indicate a moderate increase in hardware resource usage due to additional optimization techniques.

Table 5.3
Resource utilization comparison between conventional and proposed AHB-to-APB bridge

Parameter	Conventional	Proposed	% Overhead
Slice LUTs	43	60	39.5
Slice Registers	135	144	6.67
Slice	50	64	28
LUTs as Logic	43	60	39.5

The number of Slice LUTs increases from 43 to 60, representing a 39.5% increase. This increase is primarily due to the implementation of optimized FSM logic, early address decoding, and additional control mechanisms. Similarly, the number of Slice Registers increases slightly from 135 to 144, reflecting a modest overhead of 6.67%. The total number of slices used also increases, indicating additional logic required for performance optimization. While the proposed design consumes more resources, this trade-off is justified by the significant improvements in latency and operating frequency. The increase in resource utilization remains within acceptable limits for FPGA

implementation and does not impact scalability. Thus, the design achieves a balanced trade-off between performance enhancement and hardware cost.

5.6 Power Consumption Analysis

The power consumption of the proposed bridge is evaluated and compared with the conventional design, as presented in Table 5.4. The results indicate that the proposed design maintains similar power characteristics despite improvements in performance.

Table 5.4
Timing performance comparison between conventional and proposed AHB-to-APB bridge

Parameter	Conventional	Proposed
Static Power Consumption	70 mW	70 mW
Dynamic Power Consumption	09 mW	09 mW
Total Power Consumption	79 mW	79 mW

The static power consumption remains unchanged at 70 mW, as it primarily depends on FPGA characteristics rather than design complexity. The dynamic power consumption is also maintained at 9 mW, indicating that the additional logic introduced does not significantly increase switching activity.

The total power consumption of the system remains at 79 mW, demonstrating that the proposed optimizations do not incur additional power overhead. This is achieved through efficient design techniques such as fine-grain clock gating, which reduces unnecessary switching activity.

The ability to improve performance without increasing power consumption highlights the efficiency of the proposed design and makes it suitable for low-power embedded applications.

CHAPTER 6

Conclusions

This work presents a low-latency and power-efficient FPGA-based AHB-to-APB bridge designed and implemented using Verilog HDL. The primary objective of improving communication efficiency between high-speed AHB and low-power APB buses has been successfully achieved through architectural and control-level optimizations. The proposed design incorporates key techniques such as early address decoding, an optimized FSM with transaction overlap capability, fine-grain clock gating, and an efficient data-path structure. These improvements collectively enable faster protocol conversion and reduce unnecessary idle cycles. As a result, the bridge achieves a latency reduction of 25% in write operations, corresponding to a reduction of one clock cycle compared to conventional AHB-to-APB bridge designs. The timing analysis results demonstrate that the proposed design supports a higher maximum operating frequency, indicating improved timing robustness and better critical path optimization. Additionally, power analysis shows that both static and dynamic power consumption remain comparable to conventional designs, primarily due to the use of selective clock gating techniques that minimize unnecessary switching activity. From a hardware perspective, the resource utilization results indicate that the achieved latency improvements come with only minimal overhead in terms of LUTs and registers. This confirms that the design maintains an efficient balance between performance enhancement and hardware cost. Overall, the proposed AHB-to-APB bridge improves system responsiveness and throughput while preserving simplicity, scalability, and power efficiency. These characteristics make it a suitable solution for modern System-on-Chip (SoC) architectures and embedded applications where efficient interconnect design is critical.

CHAPTER 7

Future Scope

The proposed AHB-to-APB bridge demonstrates effective latency reduction and improved timing performance; however, certain limitations remain. The current design assumes fixed-latency APB transactions and does not include PREADY support, which restricts its applicability to peripherals with deterministic response times. Future work can extend the design to support variable-latency peripherals for better real-world adaptability. The validation environment is simplified using driver and scanner modules, which, while suitable for FPGA demonstration, does not fully represent complex SoC scenarios. More rigorous verification using advanced techniques such as SystemVerilog-based testbenches and coverage-driven verification can improve design robustness. Although the design achieves a one-clock-cycle latency reduction, further improvements may be explored through advanced optimization techniques such as enhanced pipelining or parallel transaction handling. Additionally, while the increase in hardware resources is moderate, further optimization can be performed to improve area efficiency. Power optimization in the current design is limited to clock gating in the address decoder. This can be extended to other modules or combined with advanced techniques such as dynamic power management for better energy efficiency. Overall, the work successfully meets its objectives and provides a strong foundation for future development. It can be extended towards scalable, multi-master SoC systems and integrated with advanced AMBA protocols, offering significant scope for further research and improvement.

References

- [1] F.-C. Yang, Y.-T. Lin, C.-F. Kao, and I.-J. Huang, “An on-chip AHB bus tracer with real-time compression and dynamic multiresolution supports for SoC,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 19, no. 4, pp. 571–584, Apr. 2011.
- [2] P. Giridhar, and P. Choudhury, “Design and Verification of AMBA AHB,” *Proc. IEEE 1st International Conference on Advanced Technologies in Intelligent Control, Environment, Computing & Communication Engineering (ICATIECE)*, Bangalore, India, Mar. 2019.
- [3] Ravikumar E., Ganesh K. B., Chethana C. Y., and Parikshith R. T. M., “Design and Verification of Advanced Microcontroller Bus Architecture (AMBA) Advanced Peripheral Bus (APB) Protocol,” *Proc. IEEE International Conference on Smart Systems for applications in Electrical Sciences (ICSSES)*, Tumakuru, India, May 2024.
- [4] S. K. K. Yadav, P. Satyanarayana, B. V. Krishna, K. R. K. Sai, Arun M., and V. G. Krishnan, “Design and Implementation of AMBA-APB Protocol for System-on-Chip (SoC) Designs,” *Proc. IEEE Third International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE)*, Ballari, India, Apr. 2024.
- [5] S. Sonal and L. M. Saini, “AMBA based AHB2APB bridge: An overview,” *Proc. IEEE International Conference on Measurement, Instrumentation, Control and Automation (ICMICA)*, Kurukshetra, India, May 2024.
- [6] Aditya, and L. M. Saini, “Overview of AMBA AHB2APB bridge for SOC Interconnects,” *Proc. IEEE International Conference on System, Computation, Automation and Networking (ICSCAN)*, PUDUCHERRY, India, Dec. 2024.
- [7] Rangeetha S., Pandithurai S., Prassath A., and Reagan C. R. A., “Design and Implementation of AHB to APB Bridge and AHB to UART Communication for SoC Integration,” *Proc. IEEE 2nd World Conference on Communication & Computing (WCONF)*, RAIPUR, India, July 2024.
- [8] K. Rawat, K. Sahni, and S. Pandey, “Design of AMBA APB bridge with reset

- controller for efficient power consumption,” *Proc. IEEE 9th International Conference on Industrial and Information Systems (ICIIS)*, Gwalior, India, Dec. 2014.
- [9] B. Karthiga, Stalin A., Saran S., Saron E. J., and Sanjay J., “Scalable and Low-Power AHB-to-APB Bridge for Reconfigurable SOC Architectures,” *Proc. IEEE 7th International Conference on Energy, Power and Environment (ICEPE)*, Sohra (Cherrapunjee), India, May 2025.
 - [10] A. Gaur, P. Sharma, and S. P. Pandey, “HDL and Timing Analysis of AMBA AHB on FPGA Platform,” *Proc. IEEE Recent Developments in Control, Automation & Power Engineering (RDCAPE)*, Noida, India, Oct. 2017.
 - [11] A. Paunikar, R. Gavankar, N. Umarikar, and K. Sivasankaran, “Design and Implementation of Area Efficient, Low Power AMBA-APB Bridge for SoC,” *Proc. IEEE International Conference on Green Computing Communication and Electrical Engineering (ICGCCEE)*, Coimbatore, India, Mar. 2014 .
 - [12] J. Romero, N. Cuevas, and E. Roa, “Energy efficient peripheral and system buses for low-area and low-power SoC applications,” *IEEE Transactions on Circuits Systems-II, Exp. Briefs*, vol. 67, no. 5, pp. 866-870, May 2020.
 - [13] N. Deshpande, and R. Sadakale, “AMBA AHB to APB Bridge Protocol Verification Using System Verilog,” *Proc. IEEE First International Conference on Advances in Electrical, Electronics and Computational Intelligence (ICAEECI)*, Tiruchengode, India, Oct. 2023.
 - [14] S. Divekar, and A. Tiwari, “Multichannel AMBA AHB with Multiple Arbitration Technique,” *Proc. IEEE International Conference on Communication and Signal Processing*, Melmaruvathur, India, Apr. 2014.
 - [15] A. Suresh, and S. Nandi, “Physical Bridge Design for High Performance to Peripheral Bus: An Open-Source Approach,” *Proc. IEEE Third International Conference on Advances in Electronics, Computers and Communications (ICAECC)*, Bengaluru, India, Dec. 2020.
 - [16] I. H. Shanavas, P. A. Reddy, M. M. Baburaj, N. C. Krishna, and Nithin S., “Design and Analysis of APB and AHB Lite Protocol,” *Proc. IEEE 7th International Conference on Design Innovation for 3 Cs Compute Communicate*

Control (ICDI3C), Karnataka, India, Nov. 2023.

- [17] S. A. Edidi, R. Marada, T. A. Khan, and Chitra E., “Improving Data Integrity with Reversible Logic-based Error Detection and Correction Module on AHB-APB Bridge,” *Proc. IEEE International Conference on Recent Advances in Electrical, Electronics, Ubiquitous Communication, and Computational Intelligence (RAEEUCCI)*, Chennai, India, Apr. 2023.
- [18] M. B. Savadatti, Vinay R., Sahil S. M. L., Shashank V., and T. J. Reddy, “Data Transfer Using AMBA Bus: An Empirical Approach,” *Proc. IEEE 2nd International Conference on Artificial Intelligence and Machine Learning Applications Theme: Healthcare and Internet of Things (AIMLA)*, Namakkal, India, Mar. 2024.
- [19] Niharika B. H., and Ramesh S., “The Configuration and Verification Analysis of AMBA-Based AHB2APB Bridge,” *Proc. IEEE 2nd International Conference on Mobile Networks and Wireless Communications (ICMNWC)*, Tumkur, Karnataka, India, Dec. 2022.